

LLMOrchestrator

LLMOrchestrator

Адзіная панэль кіравання для кожнага аўтаномнага CLI агента-кадавальніка.

Зводка

LLMOrchestrator — гэта аўтаномны, шматразовы Go модуль для стварэння, кіравання і зносін з аўтаномнымі CLI агентамі (OpenCode, Claude Code, Gemini, Junie, Qwen Code) праз гібрыдны пратакол канал+файл, з індывідуальнымі засцерагальнікамі для кожнага агента, зменнай сістэмай выбару пастаўшчыкоў, аддзеленым абстракцыйным слоём інтэрнацыяналізацыі і гарантыямі супраць ілжывых тэстаў.

Кароткае апісанне

Шматразовы Go модуль, які забяспечвае адзіны інтэрфейс для стварэння і кіравання некалькімі LLM-агентамі на базе CLI праз гібрыдны пратакол канал+файл. Бяспечнае для шматпатокавасці пуліраванне агентаў з засцерагальнікамі і выбіральнымі стратэгіямі маршрутызацыі, наўмысна незалежнае ад спажыўца, са зменнай сістэмай перакладу.

Падрабязнае апісанне

LLMOrchestrator — гэта агульная інфраструктура для аркестрацыі аўтаномных CLI агентаў-кадавальнікаў — той “водна-камунальны” пласт, які патрэбен кожнай шмат-агентнай сістэме і які звычайна дрэнна перавынаходзяць. Замест таго, каб кожны праект паўтараў рэалізацыю запуску працэсаў, фарматавання паведамленняў і парсінгу вынікаў для такіх інструментаў, як OpenCode, Claude Code, Gemini CLI, Junie і Qwen Code, ён прапануе адзіны інтэрфейс Agent, бяспечны для шматпатокавасці пул AgentPool і MultiProviderPool, які аб’ядноўвае агентаў ад некалькіх пастаўшчыкоў пад

адзіным фасадом. Маршрутызацыя з'яўляецца зменнай праз AgentSelector — кругавая чарга, якая прапускае пастаўшчыкоў, што не адпавядаюць патрабаванням, або чарговасць з прыярытэтамі і рэзервовым варыянтам — такім чынам, спосаб размеркавання задач — гэта палітыка, якую вы выбіраеце, а не жорстка запраграмаванае дапушчэнне. Кожны канкрэтны агент — гэта тонкі адаптар над агульным BaseAdapter, які кантралюе ўвесь жыццёвы цыкл працэсу: запуск з настройкай канала, карэктнае спыненне праз SIGTERM, а затым SIGKILL, перазапуск і праверку жыццяздольнасці — усё тое складанае і схільнае да памылак, што вырашана аднойчы.

Камунікацыя наўмысна гібрыдная, падбіраючы транспарт пад задачу. Канальны транспарт перадае радкова-раздзелены JSON з абмежаваннем часу чакання адказу і лімітам даўжыні адказу для хуткіх інтэрактыўных паведамленняў, у той час як файлы транспарт выкарыстоўвае асобныя каталогі ўваходных/выходных даных і агульнай прасторы для вялікіх ці доўгатэрміновых артэфактаў, якія не павінны захоўвацца ў канале. Стойкасць да збояў — не дадатковая думка, а структурны прынцып: індывідуальны засцерагальнік агента адкрываецца пасля трох паслядоўных збояў з 60-секундным астуджэннем перад паўадчыненым зондаваннем, а фоновы манітор здароўя рэгулярна правярае агентаў, каб зламаны агент мог аднавіцца, не чакаючы ўваходнага трафіку. Атрыманне агента з пула блакуецца на ўмовавай зменнай замест спажывання працэсарнага часу ў бясконцым чаканні, а парсер адказаў не мае ўнутранага стану і бяспечны для паралельнага выкліку. Модуль строга аддзелены — ніякія дэталі спажываюцца не дапускаюцца ўнутр — і кожны радок для карыстальніка праходзіць праз зменную сістэму інтэрнацыяналізацыі Translator, з NoopTranslator, які вяртае ідэнтыфікатары паведамленняў у нязменным выглядзе, каб адсутнасць перакладу было відавочна, а не схавана.

Чаму мы яго стварылі

Кожнай шмат-агентнай сістэме патрэбна надзейна запуская і падтрымліваць зносіны з CLI агентамі. Паўторнае вырашэнне пытанняў запуску, фарматавання, парсінгу і апрацоўкі збояў для кожнага праекта — неэфектыўна і схільна да памылак. LLMOrchestrator цэнтралізуе гэтыя функцыі ў адным аддзеленым, шматразовым модулі, чыё спецыялізаванае прызначэнне робіць яго паўторна выкарыстоўвальным — і гэтая паўторная выкарыстоўвальнасць знікае, калі ўнутр трапляюць дэталі канкрэтнага спажываюцца.

Змест

Чаму гэта змяняе правілы гульні

Гэта ператварае “кіраванне арміяй разнастайных агентаў CLI” з індывідуальнага інжынернага кашмару для кожнага праекта ў просты імпорт бібліятэкі — пулінг, абарона ад перагрузак, кіраванне жыццёвым цыклам і гнуткая маршрутызацыя ўжо вырашаны і правераны на практыцы. А паколькі антыблефавыя тэсты правяраюць сапраўдную сістэму ад пачатку да канца, а не абмяжоўваюцца “кампілюецца — і добра”, вы атрымліваеце абстракцыю, якой можна даваць у умовах канкурэнтнасці і збояў, а не проста нешта, што добра выглядае на дыяграме.

Што новага

- **Гібрыдны пратакол каналаў і файлаў** — інтэрактыўная хуткасць (радкі JSON праз stdin/stdout, дэдлайны чытання, абмежаванні адказаў) і надзейныя файлы абмен (inbox/outbox/shared) для вялікіх артэфактаў, каб ніколі не прыходзілася выбіраць паміж затрымкай і надзейнасцю.
- **Пул з некалькімі пастаўшчыкамі і гнуткімі селектарамі** — адзіны інтэрфейс для розных пастаўшчыкаў CLI з маршрутызацыяй па чарзе ці згодна прыярытэтам, якая выбіраецца як палітыка, а не жорстка прапісваецца.
- **Абарона ад перагрузак для кожнага агента + фоновы манітор здароўя** — аўтаматычнае зніжэнне нагрузкі і аднаўленне (3 збоі → 60 с адкрытага стану → паўадкрытая праверка), каб няздавальны агент ізаляваўся, а потым ціха вяртаўся ў строй без ручнога ўмяшання.
- **Пул без актыўнага чакання** — Acquire блакуецца на `sync.Cond`, пакуль не вызваліцца адпаведны здаровы агент ці не будзе адменены кантэкст, таму чаканне не спажывае працэсарныя рэсурсы.
- **Жорсткае раздзяленне + антыблефавая інтэрнацыяналізацыя** — NoopTranslator вяртае ідэнтыфікатары паведамленняў без змен, таму адсутнасць перакладу немагчыма прапусціць замест таго, каб маўкліва замяняць яго на пусты радок.
- **Бяспека па змоўчанні** — дазволены спіс шляхоў да бінарнікаў выключае інтэрпрэтацыю каманд і, такім чынам, паверхню для ін’екцый, падмацаваную абаронай ад пераходу па шляхах, абмежаваннем адказу ў 1 MiB супраць некантраляванага вываду і маскіроўкай ключоў API у логах.
- **Антыблефавы фрэймворк Challenge** — рэальныя кругавыя тэсты на дыску/JSON/парсары ў пяці лакальных парным мутацыйным шлюзам, які павінен вяртаць ненулявы код выхаду, калі функцыя зламаная, — тэст, які даказвае, што сістэма можа сапраўды праваліцца.

Найбольшыя тэхнічныя выклікі і як мы іх пераадолелі

- **Надзейны ўвод-вывад агенцкага працэсу.** Зносіны з запушчаным працэсам CLI аказваюцца нечакана складанымі; вырашана праз гібрыдны транспарт каналаў і файлаў, дакладна вызначаны кантракт паведамленняў/парсара, каб абодва бакі ўзгаднілі фармат перадачы, і BaseAdapter, які цэнтралізуе ўвесь жыццёвы цыкл працэсу, уключаючы карэктнае завяршэнне па SIGTERM з эскалацыяй да SIGKILL у выпадку тайм-аўту.
- **Канкурэнтнасць без актыўнага чакання.** Вырашана праз пул агентаў AgentPool на аснове м'ютакса і ўмоўнай пераменнай, дзе Acquire чакае, пакуль не вызваліцца агент з патрэбнымі магчымасцямі, у спалучэнні з бязстароннім, бязэфектыўным парсарам, бяспечным для выкліку з некалькіх гаруцін адначасова.
- **Ізаляцыя збояў пастаўшчыкаў.** Вырашана так, каб адзін няспраўны пастаўшчык не мог пацягнуць за сабой іншых: абарона ад перагрузак для кожнага агента абмяжоўвае радыус паражэння, а фоновая гаруціна манітора здароўя забяспечвае аднаўленне нават тады, калі няма запытаў, якія маглі б яго справакаваць.
- **Доказ карэктнасці, а не проста кампіляцыі.** Вырашана праз фрэймворк Challenge: дзясяткі інварыянтаў на англійскай, сербскай, японскай, іспанскай і нямецкай мовах, якія правяраюць сапраўдную сістэму, а таксама парны мутацыйны шлюз (LLMORCH_MUTATE_RUNNER=1 павінен праваліцца → абгортка вяртае код 99), які наўмысна ламае функцыю, каб даказаць, што сам шлюз не з'яўляецца блефам.
- **Лакалізацыя без маўклівых збояў.** Вырашана праз шво NoopTranslator з вяртаннем ідэнтыфікатараў без змен і ін'екцыю перакладчыка для кожнага спажыўца, каб прабелы ў перакладах заўсёды былі бачныя, а не затушоўваліся.

Змесціва

Тэхналагічны стэк

- **Go (1.25)** — абраны дзякуючы першакласнай канкурэнтнасці і чыстаму кантролю працэсаў, што менавіта патрабуецца для аркестрацыі жывых агенцкіх працэсаў; рэалізуе модуль, яго агенцкія адаптары, транспартныя сродкі і парсер.
- **Go stdlib толькі (+ testify, yaml.v3)** — свядомы выбар, каб мінімізаваць паверхню залежнасцей і не падцягваць ніякіх SDK LLM, дзякуючы чаму модуль застаецца лёгкай і

прыдатным для ўбудавання ў любы кансуматар без дадатковага вендарскага багажу.

- **Транспарт праз каналы (JSON-lines праз stdio)** — абраны для хуткай інтэрактыўнай перадачы паведамленняў, узмоцнены дэдлайнамі чытання і абмежаваннямі на даўжыню адказу, каб завіслы ці выйшаў з-пад кантролю агент не мог заблакаваць кліента.
- **Файлавы транспарт (inbox/outbox/агульны)** — абраны для надзейнага абмену вялікімі артэфактамі ў межах сесіі, дзе каналы былі б няправільным інструментам.
- **sync.Mutex/sync.Cond** — абраны для рэалізацыі блакавальнага, справядлівага атрымання агентаў з пула без актыўнага чакання.
- **Ахоўны выключальнік + HealthMonitor** — абраны разам, каб забяспечыць рэзістэнтнасць кожнага агента і актыўнае аднаўленне, а не толькі выяўленне збояў.
- **pkg/i18n Translator** — абраны як развязаны лакалізацыйны шво, які трымае спецыфічныя для кансуматараў радкі па-за асноўным ядром.
- **Тэставы фрэймворк (challenges/runner) + Makefile (test -race, fuzz, cover)** — абраны для непрымання на веру і праверкі на аснове доказаў, уключаючы выяўленне гонкі і фазінг парсера, каб карэктнасць была паказана ў адверсных умовах, а не проста меркавалася.

Стан і заўвагі аб шчырасці

- **Стан: бэта.** Развязаны модуль шматразовага выкарыстання, які ўжываецца як сабмодуль у некалькіх праектах Helix/vasic. **Ліцэнзія: Apache-2.0**; рэпазіторый GitHub з'яўляецца публічным.
- Метаданыя мадэлі паступаюць з LLMsVerifier праз HelixQA; гэты модуль не імпартае LLMsVerifier/VisionEngine/DocProcessor непасрэдна. Стэкі, згаданыя ў CLAUDE.md бацькоўскага дадатка (Gin/PostgreSQL і г.д.), апісваюць helix_code, а не гэты модуль.

Прыярытэтны ўзровень: Helix-першасны (кластар LLM-інфраструктуры — развязаны модуль шматразовага выкарыстання). Ранжыруецца пасля HelixTrack.